

ADBS UNIT-3

1. Introduction to Stored Programming

- ❑ **Stored programming allows SQL code (like procedures, functions, triggers) to be stored and executed inside the MySQL server**, reducing repeated coding and improving performance.
- ❑ It supports **procedural logic** such as variables, conditions, and loops, helping move business logic closer to the data for better efficiency.

2. Why Stored Programming

- **Improves performance** by reducing repeated client-server communication.
- **Reuses code** because logic is stored once and used many times.
- **Improves security** by controlling access through stored programs.
- **Ensures consistency** since the same logic runs for all users.

3. Types of Stored Programs in MySQL

- **Stored Procedures** – SQL code blocks that perform tasks and can take parameters.
- **Stored Functions** – Similar to procedures but must return a single value.
- **Triggers** – Code that runs automatically when INSERT, UPDATE, or DELETE happens on a table.

4. Stored Procedures

A stored procedure is a **named SQL program saved in the database** that can be executed anytime using CALL. It can have **IN, OUT, INOUT** parameters and can include logic like conditions and loops.

Syntax

```
DELIMITER $$
CREATE PROCEDURE proc_name()
BEGIN
    -- SQL statements
END$$
DELIMITER ;
```

Simple Example

```
DELIMITER $$
CREATE PROCEDURE showAllEmployees()
BEGIN
    SELECT * FROM employees;
END$$
DELIMITER ;
```

Call it:

```
CALL showAllEmployees();
```

5. Creating and Calling Stored Procedures

Syntax:

```
DELIMITER $$
CREATE PROCEDURE proc_name()
BEGIN
    -- SQL statements
END$$
DELIMITER ;
```

Calling a Stored Procedure

```
CALL proc_name();
```

Simple Example

1. Create Procedure

```
DELIMITER $$
CREATE PROCEDURE showEmployees()
```

```
BEGIN
  SELECT * FROM employees;
END$$
DELIMITER ;
```

2. Call Procedure

```
CALL showEmployees();
```

6. Stored Procedure Without Parameters

A stored procedure without parameters is a procedure that **does not take any input or return values**. It simply runs the SQL statements written inside it.

Syntax

```
DELIMITER $$
CREATE PROCEDURE proc_name()
BEGIN
  -- SQL statements
END$$
DELIMITER ;
```

Example

```
DELIMITER $$
CREATE PROCEDURE listEmployees()
BEGIN
  SELECT * FROM employees;
END$$
DELIMITER ;
```

Calling

```
CALL listEmployees();
```

7. How to Code Input and Output Parameters

Type	Direction	Usage
IN	Input to procedure	Default; passed by value
OUT	Output from procedure	Returns data to the caller
INOUT	Input and output both	Accepts and returns values

MySQL allows three parameter types in stored procedures: **IN**, **OUT**, and **INOUT**. They are used to **pass values into** the procedure and **return values back** to the caller.

Syntax

```
CREATE PROCEDURE proc_name(IN in_param INT, OUT out_param INT)
BEGIN
  -- SQL statements
END;
```

Simple Example

```
DELIMITER $$
CREATE PROCEDURE getSalary(IN emp_id INT, OUT sal INT)
BEGIN
  SELECT salary INTO sal FROM employees WHERE id = emp_id;
END$$
```

DELIMITER ;

Calling

```
CALL getSalary(102, @sal);  
SELECT @sal;
```

8. Simple Stored Procedure with an IN Parameter

An **IN parameter** is used to **pass a value into** the stored procedure. The procedure uses this value but **cannot change it**.

Example

Task: Display employee details for a given employee ID.

```
DELIMITER $$  
CREATE PROCEDURE getEmployee(IN emp_id INT)  
BEGIN  
    SELECT * FROM employees WHERE id = emp_id;  
END$$  
DELIMITER ;
```

Calling

```
CALL getEmployee(102);
```

9. Stored Procedure with IN and OUT Parameters

A stored procedure can take an **IN parameter** (input value) and return a result using an **OUT parameter**.

Example

Task: Get the salary of an employee and return it as output.

```
DELIMITER $$  
CREATE PROCEDURE getEmpSalary(IN emp_id INT, OUT emp_salary INT)  
BEGIN  
    SELECT salary INTO emp_salary  
    FROM employees  
    WHERE id = emp_id;  
END$$  
DELIMITER ;
```

Calling

```
CALL getEmpSalary(102, @sal);  
SELECT @sal;
```

10. How to Declare and Set Variables in MySQL Stored Programs In MySQL

1. **Variables must be declared inside a BEGIN...END block** using DECLARE and always **before any SQL statements**.
2. Variables are **local to the procedure** and used to store temporary values for logic or calculations.
- 3.

Syntax

```
DECLARE var_name datatype;  
SET var_name = value;
```

Simple Example

```
DELIMITER $$  
CREATE PROCEDURE getSalary()  
BEGIN  
    DECLARE sal INT;  
    SELECT salary INTO sal FROM employees WHERE id = 102;  
END$$  
DELIMITER ;
```

Key Points

- DECLARE comes **before** SET or SELECT statements.
- SELECT ... INTO is used to store a query result into a variable.
- Variables exist **only inside** the procedure block.

11. How to Display Data in Stored Programs

In stored programs, data is displayed by **retrieving column values into local variables** using SELECT ... INTO.

This lets you store a single row's data into variables and then use them for logic or output.

Syntax

```
SELECT column_name INTO variable_name
FROM table_name
WHERE condition;
```

Simple Example

```
DECLARE empName VARCHAR(50);
```

```
SELECT name INTO empName
FROM employees
WHERE id = 101;
```

Key Points

- SELECT ... INTO must return **exactly one row**.
- Used to store table values into variables inside procedures.
- Variables must be declared before using them.

12. How to Work with User Variables

User variables are **session-level variables** in MySQL that start with @.

They are used to **store values temporarily** and **pass data between SQL statements**.

Syntax

```
SET @var = value;
SELECT @var;
```

Example

```
SET @count = 10;
SELECT @count;
```

Key Points

- Start with @
- Exist **only for the current session**
- Can be used in **SELECT, WHERE, UPDATE**, etc.

13. How to Set a Default value for a Parameter

You can give a parameter a **default value** so the user may call the procedure **with or without passing** that parameter. If no value is given, MySQL uses the default.

Syntax

```
CREATE PROCEDURE proc_name(IN p1 INT DEFAULT 100)
BEGIN
  -- SQL statements
END;
```

Simple Example

```
DELIMITER $$
CREATE PROCEDURE getEmployees(IN dept_id INT DEFAULT 10)
BEGIN
  SELECT * FROM employees WHERE department = dept_id;
END$$
```

DELIMITER ;

Calling

CALL getEmployees(); -- uses default 10

CALL getEmployees(20); -- uses 20

14. How to Code IF Statements

The **IF statement** in MySQL stored programs is used to run different SQL code based on conditions, just like in normal programming.

Syntax

```
IF condition THEN
    statements;
ELSEIF condition2 THEN
    statements;
ELSE
    statements;
END IF;
```

Simple Example

```
IF marks >= 90 THEN
    SET grade = 'A';
ELSEIF marks >= 75 THEN
    SET grade = 'B';
ELSE
    SET grade = 'C';
END IF;
```

Key Points

- Used **inside** stored procedures/functions.
- Can have **IF**, **ELSEIF**, and **ELSE** parts.
- Helps execute code based on conditions.

15. Stored Procedure to Insert a Row

A stored procedure can be used to **insert a new record** into a table using INSERT. Parameters are usually passed to insert dynamic values.

Syntax

```
CREATE PROCEDURE proc_name(IN p1 datatype, IN p2 datatype, ...)
BEGIN
    INSERT INTO table_name(col1, col2, ...)
    VALUES(p1, p2, ...);
END;
```

Simple Example

```
DELIMITER $$
CREATE PROCEDURE addEmployee(
    IN emp_id INT,
    IN emp_name VARCHAR(50),
    IN emp_salary INT
)
BEGIN
    INSERT INTO employees(id, name, salary)
    VALUES(emp_id, emp_name, emp_salary);
END$$
DELIMITER ;
```

Calling

CALL addEmployee(105, 'John', 45000);

16. How to Validate Parameters and Raise Errors

In MySQL stored procedures, you can **check (validate) input parameters** using IF conditions and **raise custom errors** using the SIGNAL statement.

Syntax

```
IF condition THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error message';
END IF;
```

Simple Example

```
IF dept_id < 1 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Invalid Department ID';
END IF;
```

Key Points

- IF is used to **validate** parameters.
- SIGNAL is used to **raise an error** with a custom message.
- SQLSTATE '45000' is a generic code for user-defined errors.

17. How to Code CASE Statements

The **CASE statement** is used in stored programs when you need to check **multiple conditions** and run different code for each condition.

It works like a **switch-case** in other programming languages.

Syntax

```
CASE
    WHEN condition1 THEN statements;
    WHEN condition2 THEN statements;
    ELSE statements;
END CASE;
```

Simple Example

```
CASE
    WHEN marks >= 90 THEN SET grade = 'A';
    WHEN marks >= 75 THEN SET grade = 'B';
    ELSE SET grade = 'C';
END CASE;
```

Key Points

- Used inside procedures/functions.
- Checks multiple conditions in sequence.
- ELSE runs if none of the conditions match.

18. How to Work With Dynamic SQL

Dynamic SQL is used when the **SQL query must be built at runtime**, such as when table names, column names, or conditions are not known in advance.

MySQL uses **PREPARE**, **EXECUTE**, and **DEALLOCATE PREPARE** to run dynamic SQL.

Syntax

```
SET @sql = 'your SQL query here';
PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
```

Simple Example

Task: Select all rows from a table whose name is given at runtime.

```
SET @sql = 'SELECT * FROM employees';
PREPARE stmt FROM @sql;
EXECUTE stmt;
```

```
DEALLOCATE PREPARE stmt;
```

Key Points

- Build query as a **string** → store in a user variable.
- Use **PREPARE** to compile.
- Use **EXECUTE** to run.
- Use **DEALLOCATE** to release.

19. How to Code Loops

Loops in MySQL stored programs are used to **run a block of code repeatedly**. MySQL supports **three types of loops**: **WHILE**, **REPEAT**, and **LOOP**.

1. WHILE Loop

Runs **while the condition is true**.

Syntax

```
WHILE condition DO  
    statements;  
END WHILE;
```

Example

```
WHILE i <= 10 DO  
    SET i = i + 1;  
END WHILE;
```

2. REPEAT Loop

Runs the loop **at least once** and stops when condition becomes true.

Syntax

```
REPEAT  
    statements;  
UNTIL condition END REPEAT;
```

Example

```
REPEAT  
    SET i = i + 1;  
UNTIL i > 10 END REPEAT;
```

3. LOOP (with LEAVE)

A general loop; must use **LEAVE** to exit.

Syntax

```
loop_label: LOOP  
    statements;  
    LEAVE loop_label;  
END LOOP;
```

Example

```
myloop: LOOP  
    SET i = i + 1;  
    IF i > 10 THEN  
        LEAVE myloop;  
    END IF;  
END LOOP;
```

Key Points

- **WHILE** → condition checked first
- **REPEAT** → runs at least once
- **LOOP** → needs **LEAVE** to break

20. Comparison of All Loop Types

Loop Type	When Condition is Checked	Runs at Least Once?	How It Ends	Best Use
WHILE	Before entering the loop	✗ No	When condition becomes false	When number of iterations is known
REPEAT	After the loop body	Yes	When condition becomes true	When loop must run at least once
LOOP	No automatic check	✗ No	When LEAVE is executed	Complex loops with custom exit logic

Short Summary

- **WHILE:** Condition first → runs only if true.
- **REPEAT:** Runs once → stops when condition true.
- **LOOP:** No condition → must use LEAVE to exit.

21. Cursor in MySQL

A **cursor** is used in stored programs to **process query results row-by-row** instead of all at once. It is helpful when you need to loop through multiple rows and perform actions on each row.

Steps to Use a Cursor

1. **Declare the cursor** for a SELECT query.
2. **Open** the cursor.
3. **Fetch** each row into variables (usually inside a loop).
4. **Close** the cursor.
5. Use a **handler** to detect end of result set.

Syntax

```
DECLARE done INT DEFAULT 0;
DECLARE curName CURSOR FOR SELECT column FROM table;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
OPEN curName;
```

```
read_loop: LOOP
    FETCH curName INTO var1;
    IF done = 1 THEN
        LEAVE read_loop;
    END IF;
    -- process data
END LOOP;
CLOSE curName;
```

Simple Example (Display all employee names)

```
DECLARE done INT DEFAULT 0;
DECLARE empName VARCHAR(50);

DECLARE empCursor CURSOR FOR SELECT name FROM employees;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
OPEN empCursor;
```

```
read_loop: LOOP
    FETCH empCursor INTO empName;
    IF done = 1 THEN
        LEAVE read_loop;
    END IF;
    SELECT empName;
END LOOP;

CLOSE empCursor;
```

22. How to Use Multiple Condition Handlers

In MySQL stored programs, you can use **multiple condition handlers** to handle **different types of errors or events** separately.

Each handler tells MySQL **what to do when a specific condition occurs**, such as NOT FOUND, duplicate key, or custom errors.

Syntax

```
DECLARE CONTINUE HANDLER FOR condition1 SET var1 = value;
```

```
DECLARE CONTINUE HANDLER FOR condition2 SET var2 = value;
```

You can have **many handlers** inside the same procedure.

Simple Example

```
DECLARE done INT DEFAULT 0;
```

```
DECLARE dup INT DEFAULT 0;
```

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
DECLARE CONTINUE HANDLER FOR 1062 SET dup = 1; -- duplicate entry error
```

- NOT FOUND → sets done = 1
- Error **1062** (duplicate key) → sets dup = 1

Key Points

- Used to **catch different errors**.
- Each handler runs **only when its condition occurs**.
- Helps control flow and avoid procedure failure.

23. How to Drop a Stored Procedure

Dropping a stored procedure means **permanently deleting it** from the database.

Using IF EXISTS prevents errors if the procedure does not exist.

Syntax

```
DROP PROCEDURE IF EXISTS procedure_name;
```

Simple Example

```
DROP PROCEDURE IF EXISTS getEmployee;
```

Key Point

- Always use **IF EXISTS** to avoid “procedure already exists/doesn’t exist” errors.

24. Stored Functions

A **Stored Function** is a named SQL program stored in the database that **takes input, performs a calculation or query, and returns a single value**.

Functions can be used inside **SELECT, WHERE, ORDER BY**, etc., but **cannot modify tables** (no INSERT/UPDATE/DELETE).

Syntax

```
DELIMITER $$
```

```
CREATE FUNCTION func_name(param datatype)
```

```
RETURNS datatype
```

```
DETERMINISTIC
```

```
BEGIN
```

```
-- logic
```

```
RETURN value;
```

```
END$$
```

```
DELIMITER ;
```

Simple Example

```
DELIMITER $$
```

```
CREATE FUNCTION getBonus(salary INT)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
    RETURN salary * 0.10;
END$$
DELIMITER ;
```

Using the Function

```
SELECT getBonus(50000);
```

Key Points

- Must **return one value**.
- Used inside SQL statements.
- Cannot perform INSERT, UPDATE, DELETE.
- Use **RETURN** to send back the result.

25. Creating a Stored Function

A stored function is created to **take input, perform a calculation or query, and return one value**. It is defined using CREATE FUNCTION and must specify the **return type**.

Syntax

```
DELIMITER $$
CREATE FUNCTION function_name(param datatype)
RETURNS datatype
DETERMINISTIC
BEGIN
    -- statements
    RETURN value;
END$$
DELIMITER ;
```

Simple Example

```
DELIMITER $$
CREATE FUNCTION totalSalary(basic INT, bonus INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN basic + bonus;
END$$
DELIMITER ;
```

Using the Function

```
SELECT totalSalary(30000, 5000);
```

26. Dropping a Stored Function

Dropping a stored function means **permanently removing it** from the database. IF EXISTS is used to avoid errors if the function does not exist.

Syntax

```
DROP FUNCTION IF EXISTS function_name;
```

Simple Example

```
DROP FUNCTION IF EXISTS getBonus;
```

27. Trigger in MySQL

A **trigger** is a stored program that runs **automatically** when an event happens on a table — such as **INSERT, UPDATE, or DELETE**.

Triggers are used to **enforce rules, maintain logs, and automatically update or validate data**.

Types of Triggers

- **BEFORE** triggers → run before the DML operation.
 - **AFTER** triggers → run after the DML operation.
-

Simple Syntax

```
CREATE TRIGGER trigger_name
BEFORE|AFTER INSERT|UPDATE|DELETE
ON table_name
FOR EACH ROW
BEGIN
    -- statements
END;
```

Simple Example

```
CREATE TRIGGER log_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO emp_log(emp_id, action)
    VALUES (NEW.id, 'Inserted');
END;
```

28. Trigger in MySQL –After Trigger Example

An **AFTER trigger** runs **after** an INSERT, UPDATE, or DELETE operation is successfully completed. It is mainly used for **logging**, **auditing**, or **recording changes**.

Example (After Insert Trigger)

This trigger inserts a log entry **after** a new employee is added.

```
CREATE TRIGGER after_emp_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log(emp_id, action, log_time)
    VALUES (NEW.id, 'Inserted', NOW());
END;
```

Explanation

- AFTER INSERT → trigger runs **after** inserting a row into employees.
- NEW.id → refers to the newly inserted employee's ID.
- A log entry is stored in employee_log table.

29. MySQL – Before Trigger Example

A **BEFORE trigger** runs **before** an INSERT/UPDATE/DELETE happens. It is used for **validation**, **modifying values**, or **checking conditions** before saving data.

Example (Before UPDATE)

```
CREATE TRIGGER before_salary_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO salary_log(emp_id, old_salary)
    VALUES (OLD.id, OLD.salary);
END;
```

Simple Explanation

- Runs **before** salary is updated.
- OLD gives the previous salary value.
- Logs the old salary into salary_log table.

30. View All Triggers in Current Database

MySQL provides a simple command to list **all triggers** present in the current database.

Syntax

```
SHOW TRIGGERS;
```

This shows a table with:

- Trigger name
- Event (INSERT, UPDATE, DELETE)
- Timing (BEFORE/AFTER)
- Table it's attached to
- Statement and other metadata

Trigger Name	Event	Table	Statement	Timing
after_employee_insert	INSERT	employees	INSERT INTO employee_log (emp_id, action, log_date) VALUES (NEW.employee_id, 'INSERTED', NOW())	AFTER
before_employee_update	UPDATE	employees	INSERT INTO employee_log (emp_id, action, log_date) VALUES (OLD.employee_id, CONCAT('OLD SALARY: ', OLD.salary), NOW())	BEFORE

Explanation

- Displays trigger name, event (INSERT/UPDATE/DELETE), timing (BEFORE/AFTER), table name, and status.

31. View Triggers for a Specific Table

To see only the triggers that belong to a **specific table**, use a WHERE condition on the table name.

Syntax

```
SHOW TRIGGERS WHERE `Table` = 'table_name';
```

Example

```
SHOW TRIGGERS WHERE `Table` = 'employees';
```

★ Sample Output

Trigger	Event	Table	Timing	Statement
before_salary_update	UPDATE	employees	BEFORE	INSERT INTO salary_log...
after_emp_insert	INSERT	employees	AFTER	INSERT INTO employee_log...

32. Drop a Trigger

Dropping a trigger means **permanently removing it** from the database.
IF EXISTS avoids errors if the trigger is not found.

Syntax

```
DROP TRIGGER IF EXISTS trigger_name;
```

Example

```
DROP TRIGGER IF EXISTS after_emp_insert;
```

33. Transactions

A **transaction** is a group of SQL statements that are executed as **one single unit**.

If all statements succeed → the changes are **committed**.

If any statement fails → the changes are **rolled back**.

Transactions ensure **data consistency, reliability, and accuracy**.

Key Commands

```
START TRANSACTION; -- begin  
COMMIT;           -- save changes  
ROLLBACK;         -- undo changes
```

Simple Example

```
START TRANSACTION;  
UPDATE accounts SET balance = balance - 500 WHERE id = 1;  
UPDATE accounts SET balance = balance + 500 WHERE id = 2;  
COMMIT;
```

34. How to Work with Savepoints

Savepoints let you **create checkpoints** inside a transaction so you can **rollback part of the work** without cancelling the entire transaction.

Syntax

```
SAVEPOINT sp1;      -- create savepoint
ROLLBACK TO sp1;    -- undo only to this point
RELEASE SAVEPOINT sp1; -- remove savepoint
```

Simple Example

```
START TRANSACTION;
UPDATE emp SET salary = salary + 1000 WHERE id = 101;
SAVEPOINT s1;
UPDATE emp SET salary = salary + 500 WHERE id = 102;
ROLLBACK TO s1; -- undo second update only
COMMIT;
```

35. How to Work with Concurrency and Locking

Concurrency means **multiple users accessing the database at the same time**.

To avoid problems like **dirty reads, lost updates, and data corruption**, MySQL uses **locking** to control access.

Basic Locking Syntax

```
SELECT * FROM table_name FOR UPDATE;    -- locks rows for writing
SELECT * FROM table_name LOCK IN SHARE MODE; -- locks rows for reading
```

Simple Explanation

- **FOR UPDATE** → other transactions cannot modify these rows.
- **SHARE MODE** → others can read but cannot update.
- Locks ensure **safe and consistent** data when many users work together.

36. The Four Concurrency Problems Locks Can Prevent

Dirty Read – Reading data that another transaction has written but not committed.

Non-Repeatable Read – Getting different values when reading the same row multiple times.

Lost Update – Two transactions overwrite each other's changes.

Phantom Read – New rows appear in a repeated query because another transaction inserted them.

37. How to Set the Transaction Isolation Level

The transaction isolation level controls **how much one transaction is protected** from the effects of other transactions.

You can set it using the SET TRANSACTION ISOLATION LEVEL command.

Syntax

```
SET TRANSACTION ISOLATION LEVEL
  READ UNCOMMITTED |
  READ COMMITTED |
  REPEATABLE READ |
  SERIALIZABLE;
```

Simple Example

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Key Point

Use this **before starting a transaction** to decide how strictly MySQL should isolate concurrent operations.

38. How to Lock Selected Rows

MySQL allows you to **lock specific rows** during a transaction so other transactions cannot change them. This ensures safe updates in multi-user environments.

Syntax

```
SELECT * FROM table_name FOR UPDATE;    -- locks rows for writing
SELECT * FROM table_name LOCK IN SHARE MODE; -- locks rows for reading
```

Simple Explanation

- **FOR UPDATE** → locks the selected rows so only your transaction can modify them.
- **SHARE MODE** → allows others to read but prevents updates.
- Used inside **START TRANSACTION** to safely update specific rows.

ALL TOPICS COVERED:

1. Stored Programs in MySQL

- Stored Procedures
- Stored Functions
- Triggers

2. Stored Procedures

(Main Concepts)

- Creating & Calling Procedures
- Procedures Without Parameters
- IN, OUT, INOUT Parameters
- Declaring & Setting Variables
- Displaying Data (SELECT ... INTO)
- User Variables (@var)
- Default Parameter Values
- IF Statements
- Insert Row Procedure
- Validate & Raise Errors (SIGNAL)
- CASE Statements
- Dynamic SQL (PREPARE, EXECUTE)
- Loops
 - WHILE
 - REPEAT
 - LOOP + LEAVE
- Cursor
 - Declare
 - Open
 - Fetch
 - Loop

- Close

- Condition Handlers

- Multiple Condition Handlers
- Drop Procedure

3. Stored Functions

- Creating Functions
- Using RETURN
- DETERMINISTIC
- Usage inside SELECT
- Dropping Functions

4. Triggers

- BEFORE Triggers
- AFTER Triggers
- Trigger Examples
- View All Triggers
- View Triggers for a Table
- Drop Trigger

5. Transactions

(Highest Priority Topics)

- START TRANSACTION
- COMMIT
- ROLLBACK
- SAVEPOINT
- Concurrency & Locking
- Concurrency Problems (4)
 - Dirty Read
 - Non-Repeatable Read
 - Lost Update
 - Phantom Read
- Isolation Levels
- Lock Selected Rows (FOR UPDATE, SHARE MODE)